

MÉTODO

- Un método es un bloque de código que solo se ejecuta cuando se le llama.

```
public void Saludar() {
```

```
    Debug.Log("Hola");
```

```
}
```

Por ejemplo: tenemos un método llamado Saludar con retorno vacío(void), donde su acción/tarea es mostrar un “Hola” en pantalla”.

MÉTODO

Esto quiere decir que cada vez que el método es llamado o invocado, va a realizar su bloque de código / lógica correspondiente.

```
//---- [Llamando/invocando el metodo Saludar()]-----  
Mensaje de Unity | 0 referencias  
private void Start()  
{  
    Saludar();  
}  
  
//--- [Método Saludar] ---  
1 referencia  
public void Saludar()  
{  
    Debug.Log("Hola");  
}
```

Si llamamos el método **Saludar()** en el **Start** dentro de nuestro código en Unity. Nos va a permitir ver el saludo en pantalla, ya que se estaría ejecutando lo que se encuentre dentro del método **Saludar()** (que en este caso sería -Debug.Log("Hola")-)

MÉTODO CON PARAMETROS

También nuestro método nos permite recibir cualquier tipo de parámetros para poder manipular cierta lógica dentro de nuestro método. Si te estás preguntando ¿Qué es un parámetro? Veamos el siguiente ejemplo para entenderlo mejor.

```
Script de Unity | 0 referencias
public class SaludarDos : MonoBehaviour
{
    //-----[Llamando/invocando el método Saludar()]-----
    // Mensaje de Unity | 0 referencias
    void Start()
    {
        string miNombre = "Gonzalo";

        Saludar();
    }

    1 referencia
    public void Saludar(string nombre)
    {
        Debug.Log("Hola: " + nombre);
    }
}
```

void SaludarDos.Saludar(string nombre)

CS7036: No se ha dado ningún argumento que corresponda al parámetro requerido "nombre" de "SaludarDos.Saludar(string)"

Como vemos en la siguiente, ahora cuando queramos invocar el método Saludar nos pide el parámetro correspondiente para poder utilizarlo.

MÉTODO CON PARAMETROS

Solo tendríamos que poner una variable de tipo `string` (ya que estamos utilizando caracteres) dentro del paréntesis de `Saludar()` y enviarle nuestro nombre a través del método saludar.

```
//-----[Llamando/invocando el método Saludar()]-----  
Mensaje de Unity | 0 referencias  
void Start()  
{  
    string miNombre = "Gonzalo";  
    Saludar(miNombre);  
}  
  
1 referencia  
public void Saludar(string nombre)  
{  
    Debug.Log("Hola:" + nombre);  
}
```

En el método `Saludar(string nombre)` recibe un parámetro “`string nombre`” y lo estamos utilizando dentro del bloque del código del método. Este parámetro, lo vamos a poder utilizar solamente dentro del método, también vamos a poder modificar su valor y no vamos a tener acceso en ninguna parte del código .

MÉTODO CON MULTIPLES PARAMETROS

```
//-----[Llamando/invocando el método Saludar()]-----  
Mensaje de Unity | 0 referencias  
void Start()  
{  
    string miNombre = "Gonzalo";  
    int miEdad = 30;  
    float miTemperatura = 36.5f;  
  
    Saludar(miNombre,miEdad,miTemperatura);  
}  
  
1 referencia  
public void Saludar(string nombre, int edad, float tempratura)  
{  
    bool esMayor = false;  
  
    if(edad >= 18)  
    {  
        esMayor= true;  
    }  
  
    Debug.Log("Nombre:" + nombre);  
    Debug.Log("Temperatura:" + tempratura);  
    Debug.Log("Edad:" + edad);  
    Debug.Log("Es mayor de Edad?" + esMayor);  
}
```

Supongamos que ahora queremos que nos salude y que nos muestre la edad, temperatura corporal y si somos mayores de edad.
El método **Saludar()** tendría que recibir como parámetro las variables nombre, edad y temperatura.

MÉTODO CON MULTIPLES PARAMETROS

```
//-----[Llamando/invocando el método Saludar()]-----  
Mensaje de Unity | 0 referencias  
void Start()  
{  
    string miNombre = "Gonzalo";  
    int miEdad = 30;  
    float miTemperatura = 36.5f;  
  
    Saludar(miNombre, miEdad, miTemperatura);  
}  
  
1 referencia  
public void Saludar(string nombre, int edad, float tempratura)  
{  
    bool esMayor = false;  
  
    if(edad >= 18)  
    {  
        esMayor= true;  
    }  
  
    Debug.Log("Nombre:" + nombre);  
    Debug.Log("Temperatura:" + tempratura);  
    Debug.Log("Edad:" + edad);  
    Debug.Log("Es mayor de Edad?" + esMayor);  
}
```

En este ejemplo utilice los distintos parámetros y aplique una pequeña lógica dentro para saber si es mayor de edad o no. Recuerden que toda lógica que se encuentre dentro del método se va a ejecutar cada vez que es instanciado/llamado. Es importante el orden de cómo enviamos los parámetros. En este caso nos pide primero el nombre, después la edad y por último la temperatura. Tengan cuidado a la hora de llamar un método con parámetros.

MÉTODO CON RETORNO

Al principio les mencione que el método Saludar es de retorno **vacío (void)**, pero qué significa realmente? Al ser vacío quiere decir que no tiene ningún tipo de retorno, sólo va a ejecutar la acción que se encuentra dentro del método sin devolver ningún tipo de valor en específico. Sin embargo, tal como los parámetros, podemos asegurar que nuestro método devuelva algún tipo de valor en específico ([string](#), [float](#), [int](#), [etc](#)).

Supongamos que ahora creamos un método llamado **Suma()** donde recibe dos números como parámetros y realice dicha suma.

```
0 referencias
public void Suma(int primerNumero, int segundoNumero)
{
    int suma = primerNumero + segundoNumero;

    Debug.Log(suma);
}
```

MÉTODO CON RETORNO

```
0 referencias  
public void Suma(int primerNumero, int segundoNumero)  
{  
    int suma = primerNumero + segundoNumero;  
  
    Debug.Log(suma);  
}
```

Podemos decir que tenga un retorno de tipo `int` (`public int Suma()`) para asegurarnos que no devuelva solamente la suma y de esa forma podríamos obtener el valor de la suma para poder guardarlo en una variable o manipularlo más adelante.

MÉTODO CON RETORNO

```
//-----[Llamando/invocando el método Saludar()]-----  
Mensaje de Unity | 0 referencias  
void Start()  
{  
    int numero_1 = 1;  
    int numero_2 = 2;  
  
    int miSuma = Suma(numero_1, numero_2);  
    Debug.Log(miSuma);  
}  
  
1 referencia  
public int Suma(int primerNumero, int segundoNumero)  
{  
    int suma = primerNumero + segundoNumero;  
  
    return suma;  
}
```

Utilizamos la palabra “**return**” para decirle al compilador que va a devolver ese valor en específico.

MÉTODO CON RETORNO

Recordemos que siempre tiene que devolver el mismo tipo que el método, en este caso es **int** (entero) y devuelve la suma que es un número entero. Tengan cuidado ya que si no devolvemos ningún valor, el compilador nos mostrará el siguiente error.

```
1 referencia
public int Suma(int primerNumero, int segundoNumero)
{
    int suma =
}
```

 int SaludarDos.Suma(int primerNumero, int segundoNumero)

CS0161: 'SaludarDos.Suma(int, int)': no todas las rutas de acceso de código devuelven un valor